

# NexaVerse Technical Architecture

## *System Design & Infrastructure Guide*

### Architecture Overview

NexaVerse is built on a modern microservices architecture deployed on Microsoft Azure. The platform leverages Azure OpenAI for AI capabilities, Azure AI Search for hybrid vector search, Cosmos DB for document storage, and Azure Blob Storage for file management. The frontend is built with React and TypeScript, while the backend uses Python FastAPI.

### Document Processing Pipeline

When a document is uploaded: (1) File stored in Azure Blob Storage, (2) Azure Document Intelligence extracts text and structure, (3) Content is chunked using token-aware algorithms with 500 token chunks and 50 token overlap, (4) Azure OpenAI generates embeddings using text-embedding-3-small model, (5) Chunks indexed in Azure AI Search with vector and keyword search capabilities.

### RAG Query Flow

User queries follow this path: (1) Query received via FastAPI endpoint, (2) Content Safety API validates input, (3) Query embedding generated, (4) Hybrid search against Azure AI Search (vector + keyword), (5) Top K results retrieved with relevance scores, (6) Context sent to GPT-4o with system prompt, (7) Response streamed to client via Server-Sent Events (SSE), (8) Citations included with chunk metadata.

### Security & Access Control

Authentication uses JWT tokens with HS256 signing. Role-based access control (RBAC) enforces permissions at three levels: API endpoints, search index filtering, and document visibility. All API calls are logged to Cosmos DB audit container. Content Safety API scans both inputs and outputs for policy violations.

### Scalability & Performance

The system auto-scales based on demand using Azure App Service. Azure AI Search handles 100+ concurrent queries with <200ms latency. Document processing is asynchronous with status polling. Response caching reduces OpenAI API costs. CDN serves static assets globally for optimal performance.